BEACON

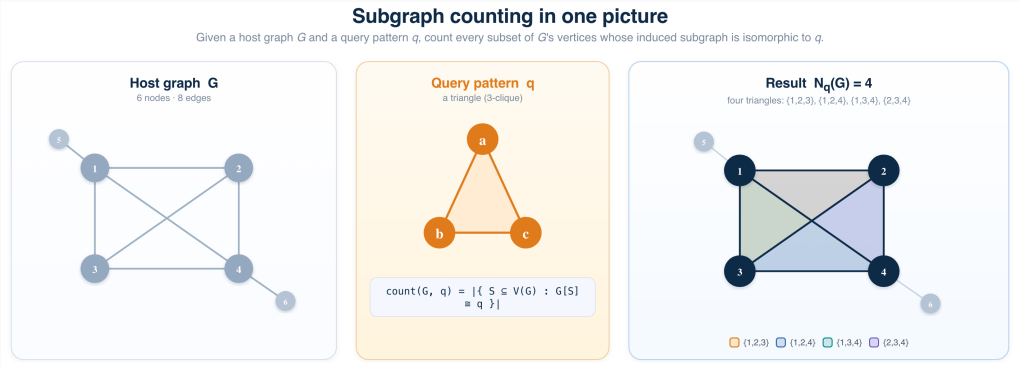
A Benchmark for Efficient and Accurate Counting of Subgraphs

Xiangju Zhu[1] **Mohammad Matin Najafi**[2] (presenting) Chrysanthi Kosyfaki[3]
Xiaodong Li[4] Reynold Cheng[1] Laks V. S. Lakshmanan[5]

ICDE 2026 | Experiment, Analysis & Benchmark Track

[1]University of Hong Kong [2]Huawei HK Research Center [3]HKUST
[4]Xiamen University [5]University of British Columbia

What is subgraph counting?



Why does it matter?

Where does subgraph counting show up in the real world?

A handful of structural motifs explain a huge fraction of valuable signal across domains.

Social networks [17]

Facebook, LinkedIn, Twitter

Triangle — tight-knit friend group, recommendation signal

Triangle counts power Facebook's **"People You May Know"**: more shared triangles around a candidate ⇒ closer community ⇒ more clicks.

Biological networks [18–20]

Protein interaction, gene regulation

Feed-forward loop — filters noise, robust regulation

Counts of FFL motifs (FGF→AKT→mTOR) reveal which gene-regulatory circuits filter cellular noise — a predictor of disease robustness.

Financial networks [21]

Bank transfers, crypto, AML

Directed *k*-cycle — money round-trip / wash trades

Counts of directed cycles of length 3–8 are the canonical AML feature for spotting layering in correspondent banking and crypto graphs (FATF 2023 typology report).

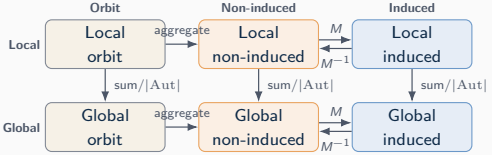
Why count and not detect?
Detection answers *"does at least one exist?"*. **Counting** answers *"how many?"* — the magnitude is what differentiates noise, normal, and anomalous behaviour at scale.

The design space — five axes, one task

Five axes define every variant:

Axis	Option A	Option B
Scope	**Global** — one count per graph	**Local** — one count per node
Edges	**Induced** — all edges in S	**Non-induced** — extras OK
Input	**Subgraph-centric** (pattern F)	**Network-centric** (all size-k)
Labels	**Typed** — coloured nodes/edges	**Untyped** (homogeneous)
Direction	**Directed** ($\rightarrow$)	**Undirected** ($-$)

Outputs are not interchangeable



Scope of this work — and why

In scope (axis values)

Axis	Choice
Direction	**Undirected**
Edges	**Induced**
Input	**Network-centric**
Labels	**Untyped** (homogeneous)

What we test $k = 3..5$ (full) + k-stars to 8; **11 methods** (4 AL + 7 ML).

On the leaderboard roadmap

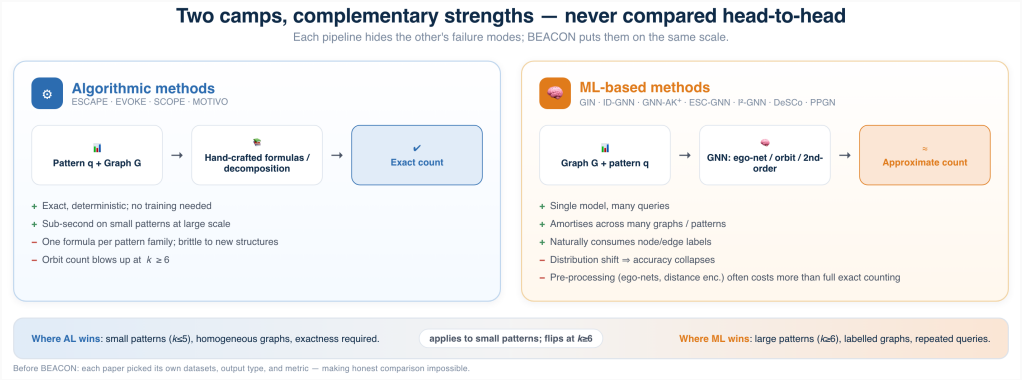
- Typed / labeled graphs (e.g. LSS [Zhao, VLDB'23])
- Directed graphs
- Dynamic / streaming
- Distributed counting

Why this scope?

- **Most valuable in practice** — Ribeiro et al. ACM CSUR'22 [8] surveys $k = 3..5$ as the practically-relevant range.
- **Common ground + tractable** — all 11 baselines support $k \leq 5$; orbits explode $58 \rightarrow 407 \rightarrow 4306$ at $k = 5, 6, 7$ (Sec. V.B).
- **Fair to AL** — labeled methods "rely on node labels as features and degenerate on unlabeled graphs" (Sec. II.e).

The landscape

Two camps — algorithmic vs ML



Algorithmic family at a glance

ESCAPE

WWW'17 • $k \leq 5$

Type: Exact, global
Idea: Degree-oriented cutting + hand-crafted formulas.

No scaling beyond k=5.

EVOKE

WSDM'20 • $k \leq 5$

Type: Exact, local (orbits)
Idea: Polynomial equations linking large to small orbits.

100× faster than prior work.

SCOPE

VLDB'24 • any k

Type: Exact, local + global
Idea: Tree decomposition + symmetry breaking.

Scales to large graphs exactly.

MOTIVO

VLDB'19 • any k

Type: Approx., global
Idea: Color coding + adaptive graphlet sampling.

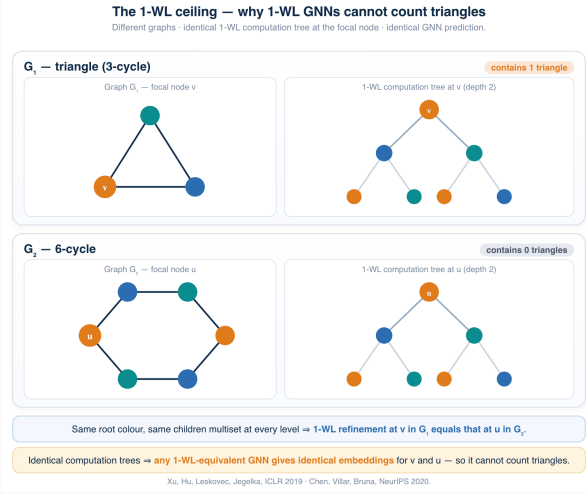
Rare/complex patterns suffer.

Strongest on small patterns at scale; orbit explosion blocks induced counting beyond $k = 5$.

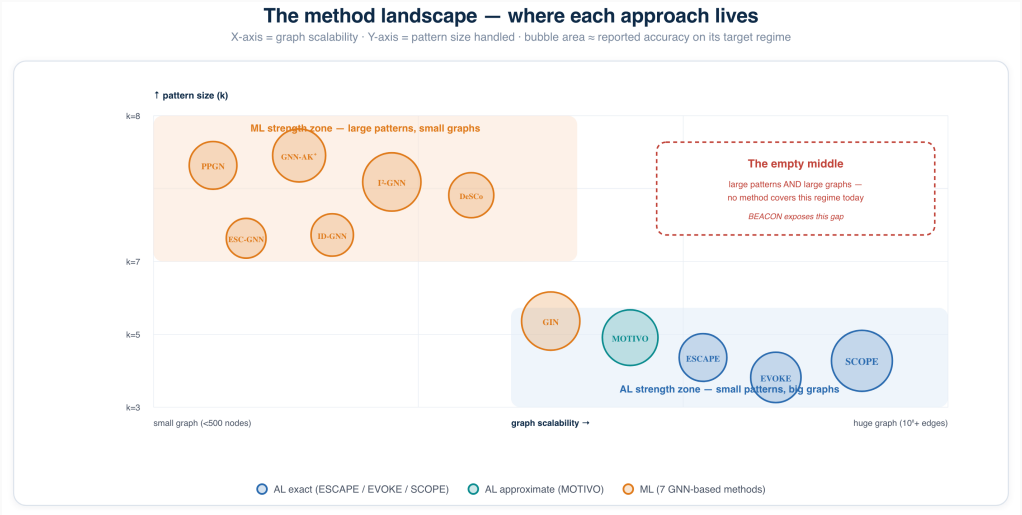
ML family + the WL barrier

Every ML method tries to break the 1-WL ceiling differently:

Method	Power	Key idea
GIN [Xu, ICLR'19]	1-WL	Injective sum aggregation
ID-GNN [You, AAAI'21]	>1-WL	Identity (ego-net colouring)
GNN-AK$^+$ [Zhao, ICLR'22]	>1-WL	Rooted subgraph encoding
ESC-GNN [Yan, KDD'24]	>1-WL	Distance-to-root embeddings
I^2-GNN [Huang, ICLR'23]	sub-GNN	Dual node IDs (counts 6-cycles)
DeSCo [Fu, WSDM'24]	sub-GNN	Canonical partition + hetero MP
PPGN [Maron, NeurIPS'19]	$\geq$3-WL	2nd-order tensor; $O(n^3)$



The full landscape — no head-to-head before BEACON



Why we built BEACON

Why a benchmark? Five blockers nobody had cleared

Five concrete blockers:

1. No shared datasets.
2. Missing ground truth — exact counts are expensive.
3. Mismatched semantics — induced vs. non-induced, local vs. global.
4. Fragile environments — GPU memory, deps, hyperparams differ.
5. Weak baselines — ML papers often compared only to outdated AL.

The pivot

We set out to build a new ML method. We ended up building the **infrastructure** that lets anyone tell whether *any* method is genuinely better.

What the community gets

Verified data Docker images
Controlled tests Public leaderboard

Usage scenarios — how the community uses BEACON

Mirroring the paper's Fig. 3(b): four researcher personas, four lifecycle stages.

Bob explore

"How can I research deeply to find weaknesses and advantages of existing works?"

1. Pull baseline Dockers
2. Tweak runtime params; ablations
3. Try different sets / scenarios

Alex design

"How can I make my new SC algorithm easy to develop and convenient for users?"

1. Encapsulate dev env
2. Mount/install code in container
3. Publish container + usage

Candy test

"I have a preliminary algorithm — how do I test scalability, robustness, accuracy, efficiency?"

1. Sample with BEACON-Sampler
2. Test across aspects
3. Find weaknesses; iterate

David ship

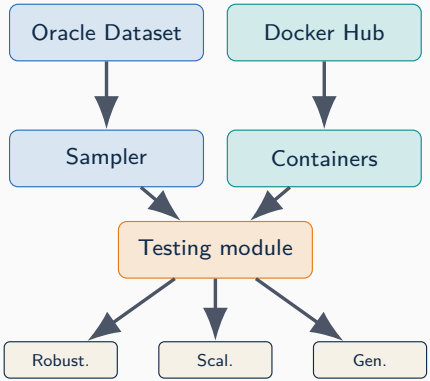
"I have my final algorithm — how do I get a fair comparison with other methods?"

1. Pull competitor Dockers
2. Run all algs on shared sets
3. Report pros / cons

→ *lifecycle*: explore → design → prototype → ship →

The BEACON benchmark

BEACON: 3 modules + 4 design principles



Four design principles (paper Sec. IV.C):

Unified & method-agnostic

Fair head-to-head between AL exact / approximate and ML.

Reproducible by construction

Docker pinned-deps + fixed CPU set; no "works on my machine".

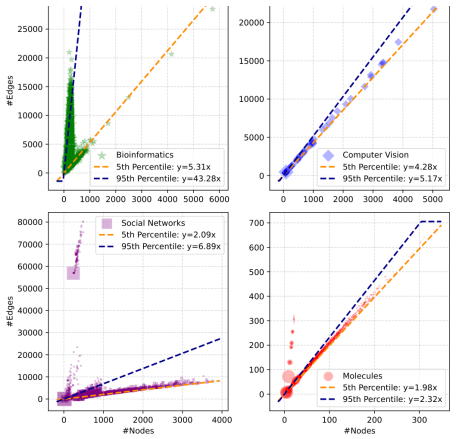
Ground-truth first

Curated Oracle (26,435 graphs) with verified counts up to k=5.

Controlled diversity & stress

BEACON-Sampler: explicit constraints on size, density, degree.

Oracle Dataset: structural diversity by design



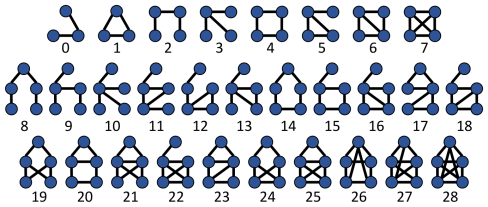
Density slope ($|E|/|V|$) differs 5× across domains.

- **26,435 graphs** — bioinformatics, vision, social, molecular.
- **All 29 patterns of size** ≤ 5 counted exactly: induced & non-induced, local & global.
- **ID-constrained ground truth** (DeSCo-style ego-net counts) shipped to seed future work.
- **120+ features** per graph (diameter, density, clustering, max local count, …).

BEACON-Sampler (PyPI: `rwdq`)

```
{nodes: [100,500], avg_deg: [1.5,5],
 local_triangle_max: [10000,20000]}
```

Benchmark protocol + how we measure



29 patterns of size 3–5, plus star patterns up to 8 nodes.

Sets 1–4	Many small graphs (≤ 500 nodes), increasing density; Set 2 drops degree cap $\rightarrow$ power-law tails
Sets 5–8	Single large graph, increasing total size
Scenarios	Zero-shot / fine-tune / full retrain

How we measure (paper Defs. 9–10)

Q-error (multiplicative):

$$Q = \max\left(\frac{\widehat{C}+1}{C+1},\ \frac{C+1}{\widehat{C}+1}\right)$$

1.0 = perfect; 10 = off by 10×. Insensitive to scale; great for rare patterns.

MAE (additive): $|\widehat{C} - C|$
Critical for large counts (up to 10^{10}).

Always report both — small MAE + large Q-err means you fail on *rare* patterns.

Zero-shot is the realistic scenario

For a static graph you count once. Needing fine-tuning defeats the purpose of using ML.

Experimental findings

Finding 1 ⏱ Algorithmic methods dominate on small-pattern speed

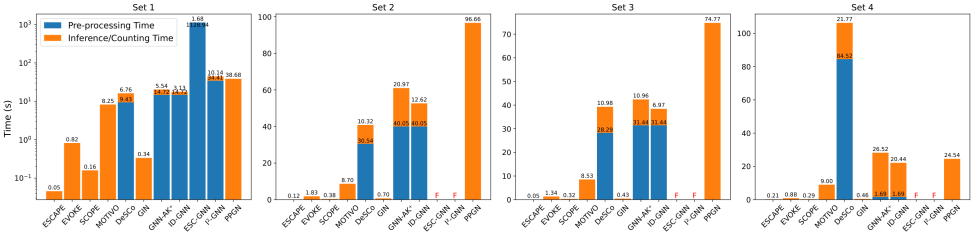


Fig. 7. Runtime time (red "F" indicates failure) for Set 1-4.

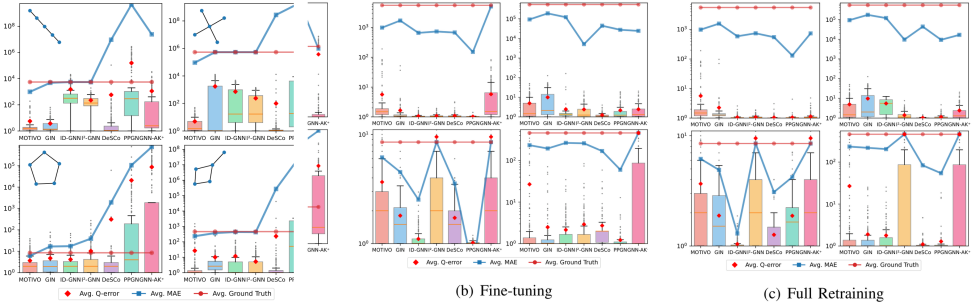
- ML pre-processing (ego-nets, distance encodings) often **exceeds full exact-method runtime**.
- "F" = OOM — ESC-GNN, I^2-GNN, PPGN fail on denser sets.
- GIN is the ML outlier: fastest ML at 4 s on Set 8.

Set 8 (single large graph):

SCOPE (exact)	114.5 s
MOTIVO	38.6 s
GIN	**4.1 s**
DeSCo	1607.0 s
Others	OOM

For $k \leq 5$ ML is the wrong tool — use optimised algorithmic indices.

Finding 2 ◎ Accuracy collapses without fine-tuning



(a) Zero-Shot (b) Fine-tuning (c) Full Retraining

Fig. 6. Accuracy under different training scenarios o Set 1.

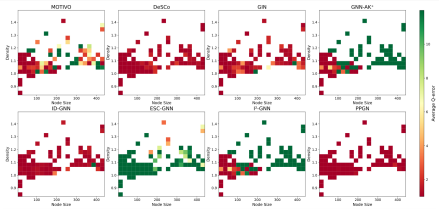
(a) Zero-shot
Most models lose to MOTIVO;
ID-GNN: Q-err < 3 on 5-cycles,
$> 10^3$ on 5-paths.
Fails on distribution shift.

(b) Fine-tuning
A few epochs $\rightarrow$ Q-err $< 10^1$.
Needs *some* ground truth.
Training data must exist.

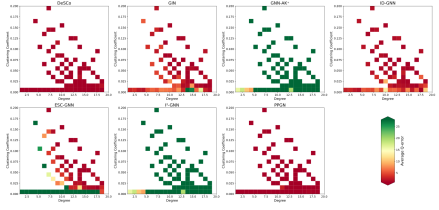
(c) Full retrain
Best accuracy but high overhead;
PPGN can *degrade* vs. fine-tune.
Defeats the point of amortising.

ML needs substantial ground-truth to generalise — partially defeating its purpose for fast estimations.

Finding 3 ⚠ Where ML systematically fails



(a) Graph Accuracy



(b) Node Accuracy

Fig. 8. Effect of graph size, density, node degree and clustering coefficient on accuracy on target 11.

Fine-tuned Q-err, target 13 ($\triangle + 2$ edges):

Algorithms	Set 1	Set 2	Set 3	Set 4
MOTIVO	108.88	469.86	2.43	**1.13**
DeSCo	1.93	**1.39**	1.13	1.21
GIN	8.33	24.63	7.69	2.23
GNN-AK$^+$	4307.59	7612.56	173.02	1438.93
ID-GNN	4.45	26.48	**1.12**	1.76
ESC-GNN	29220.07	N/A	N/A	N/A
I^2-GNN	5.01	N/A	N/A	N/A
PPGN	**1.02**	4.01	1.39	1.64

- High-degree, low-clustering graphs = worst case.
- **Set 2**: GNN-AK$^+$ Q-err hits **7612**, ESC-GNN **29220**.
- DeSCo robust via ID-constrained ego-nets.
- PPGN ($\geq$3-WL) accurate but $O(n^3)$.
- 💡 **Density heavily impacts ML; AL handles it gracefully.**

Finding 4 ML takes the lead at large patterns

TABLE VI
STAR PATTERN ACCURACY WITH MAE AND Q-ERROR FOR STAR-SHAPED PATTERNS.

Alg	3-star		4-star		5-star		6-star		7-star		8-star	
	MAE	Q-error	MAE	Q-error	MAE	Q-error	MAE	Q-error	MAE	Q-error	MAE	Q-error
MOTIVO	$1.1e+02$	1.23	$4.0e+03$	1.72	$1.1e+05$	4.97	$4.5e+07$	2.80	$7.6e+08$	6.15	$1.1e+10$	87.0
DeSCo	$1.7e+02$	1.07	$2.9e+03$	1.07	$4.4e+04$	1.25	$1.9e+07$	1.35	$5.0e+08$	6.33	$4.6e+09$	10.4
GIN	$2.8e+02$	1.10	$4.3e+03$	1.13	$3.8e+05$	2.66	$8.9e+06$	38.0	$1.3e+08$	$1.8e+03$	$1.5e+09$	$2.5e+04$
GNN-AK$^+$	$4.7e+02$	1.21	$2.0e+02$	1.01	$1.8e+04$	1.48	$5.8e+05$	1.67	$1.3e+07$	1.12	$1.4e+08$	1.54
ID-GNN	$3.4e+01$	1.01	$2.5e+03$	1.10	$7.0e+04$	1.42	$3.1e+06$	1.92	$5.3e+07$	2.56	$7.2e+08$	9.00
ESC-GNN	$2.2e+03$	10.0	$2.5e+04$	44.4	$3.1e+05$	$8.0e+02$	$8.3e+06$	$7.8e+03$	$1.2e+08$	$7.8e+03$	$1.5e+09$	$3.1e+04$
I^2-GNN	$5.5e-01$	1.00	$1.1e+01$	1.01	$1.8e+03$	2.11	$7.9e+04$	1.01	$2.9e+06$	1.54	$3.0e+07$	1.36
PPGN	$5.2e+02$	1.65	$5.1e+03$	1.20	$6.5e+03$	1.22	$4.9e+05$	1.68	$2.9e+06$	1.22	$5.9e+08$	1.62

As star-pattern size grows from 3 to 8 nodes:

- Exact AL infeasible: orbit explodes $58 \to 407 \to 4306$.
- **I^2-GNN** Q-error $\approx$1; GNN-AK$^+$ < 2.
- **MOTIVO:** Q-error 1.2$\to$**87** (3$\to$8-star).
- **GIN** (1-WL): Q-error $\approx$ 25,000 at 8-star.

The crossover point

$k \leq 5$: dominates | $k \geq 6$: often only option

BEACON locates this crossover.

Looking ahead

Synthesis — who wins, where, and why

	Small graph	Large graph
Small pattern ($k \leq 5$)	**AL exact** wins on *accuracy + speed* ESCAPE/EVOKE/SCOPE: <1 s; ML pre-proc alone exceeds it	**AL exact / approximate** wins Only GIN keeps up among ML (4 s on Set 8)
Large pattern ($k = 6-8$)	**ML wins; AL infeasible** I^2-GNN / GNN-AK$^+$ hold Q-err <2	**Only ML scales** PPGN OOMs; I^2-GNN keeps Q-err $\approx$ 1

The crossover lives at $k = 6$. BEACON is the first benchmark that actually *locates* it.

Where ML truly shines — typed graphs

The apparent paradox.

Findings 1–4 show AL beats ML on most homogeneous regimes.
So why ML at all?

The killer feature (paper Sec. V.D.b)

ML methods "*naturally support arbitrary patterns, including induced and non-induced counting*"; runtime is "*relatively insensitive to pattern structure, primarily influenced by the pattern's diameter.*"

And: they consume node/edge labels as features — AL needs new hand-crafted formulas per label combination.

Cited evidence — typed-graph SC with ML:

- **LSS** [Zhao et al. VLDB'23]
  Learned sketch for **labeled** subgraph counting.
- **Schwabe & Acosta** [PACMMOD'24]
  Cardinality estimation over **knowledge graphs** via embeddings + GNN.
- **Yu et al.** [AAAI'23]
  Learning to count isomorphisms with GNN.
- **Ego-GNNs** [ICASSP'21]
  Ego-network features for typed-node tasks.

Bridge to next steps

BEACON's homogeneous scope is the **fair common ground**; typed graphs are the next leaderboard track.

Next steps unlocked by BEACON

Hybrid AL + ML

Decompose with AL, learn the residual; "local counts of small patterns as features for larger patterns" (Sec. VI).

Helps Candy / David.

Typed / labeled track

Reactivate LSS [Zhao'23], Schwabe-Acosta [PACMMOD'24] on a labeled Oracle.

Helps Bob / David.

Directed graphs

Definition 1 currently assumes undirected; new Oracle counts needed.

Helps Alex.

Dynamic / streaming

Wackersreuther et al. MLG'10; no SOTA in benchmark yet.

Helps Candy.

Distributed counting

Zhang et al. VLDB'20 [39] is uncovered today.

Helps Alex.

Public live leaderboard

Ongoing deliverable; new methods plug in via Docker.

Helps everyone.

BEACON is open

Use it. Break it. Improve it.

- `github.com/zxj0302/MLSC`
- `pip install rwdq` — BEACON-Sampler on PyPI
- Docker Hub: pre-built images for all 11 baselines

Thank you.

- Leaderboard (coming soon): AL & ML on identical inputs
- `matin@connect.hku.hk` — questions, new datasets, contributions

Questions?